

Lista de Exercícios – Python (Funções, Strings, Loops Aninhados e Listas)

1. Palíndromos Recursivos

Implemente uma função recursiva `eh_palindromo(texto)` que receba uma string e retorne `True` se for palíndromo (desconsiderando espaços, acentos e diferenças entre maiúsculas/minúsculas) e `False` caso contrário.

Exemplo:

Entrada: "Socorram-me subi no onibus em Marrocos"

Saída: True

2. Estatísticas de Texto

Crie uma função `estatisticas_texto(texto)` que receba uma string e retorne uma lista com:

- número de palavras,
- número de caracteres,
- tamanho médio das palavras,
- a palavra mais longa.

Exemplo:

Entrada: "Python é divertido"

Saída:

```
[[ 'palavras', 'caracteres', 'tamanho_medio', 'mais_longa'], [3, 17, 5.6, 'divertido']]
```

3. Codificação Cíclica (Cifra de César)

Escreva uma função `cifra_cesar(texto, k)` que receba uma string e um número inteiro `k`.

- Cada letra deve ser deslocada `k` posições no alfabeto (**circular**).
- Não altere espaços nem pontuação.
- Aceite tanto maiúsculas quanto minúsculas.

Exemplo:

Entrada: ("Abacaxi", 3)

Saída: "Dedfdal"

4. Substring Única

Implemente uma função `maior_substring_unica(texto)` que encontre a maior substring sem caracteres repetidos.

Exemplo:

Entrada: "abcabcbb"

Saída: "abc"

Entrada: "bbbbbb"

Saída: "b"

5. Ordenação de Palavras por Critério Personalizado

Escreva uma função `ordenar_palavras(frase)` que:

1. Separe a frase em palavras,
2. Ordene as palavras primeiro pelo tamanho e depois em ordem alfabética.

Exemplo:

Entrada: "banana uva abacaxi pera morango"

Saída: ["uva", "pera", "banana", "morango", "abacaxi"]

6. Gerador de Anagramas de Palavras

Crie uma função `geraAnagramas(p)` que retorne uma lista com TODOS os anagramas de uma palavra. Ignore acentos e diferenças de maiúsculas/minúsculas. Dica: o número de anagramas será o fatorial do número de diferentes letras.

9. Contador de Vogais Únicas

Escreva uma função `vogais_unicas(texto)` que conte quantas vogais distintas aparecem na string.

Exemplo:

Entrada: "Programacao"

Saída: 3

10. Compactação de Strings (Run-Length Encoding)

Implemente uma função `compactar(texto)` que comprima a string substituindo sequências de caracteres repetidos pelo caractere seguido da contagem.

Depois, implemente `descompactar(texto)` que faça o processo inverso.

Exemplo:

Entrada: "aaabbccccc"

Saída: "a3b2c4"

Entrada: "a3b2c4"

Saída: "aaabbccccc"

11. Matriz Identidade

Crie uma função `matriz_identidade(n)` que receba um número inteiro `n` e retorne uma lista de listas representando uma matriz identidade de tamanho `n`.

Exemplo:

Entrada: 3

Saída: `[[1,0,0],[0,1,0],[0,0,1]]`

12. Transposição de Matriz

Implemente `transpor(matriz)` que receba uma matriz (lista de listas) e retorne sua transposta.

Exemplo:

Entrada: `[[1,2,3],[4,5,6]]`

Saída: `[[1,4],[2,5],[3,6]]`

13. Multiplicação de Matrizes

Escreva uma função `multiplicar_matrizes(A, B)` que multiplique duas matrizes (validando dimensões) e retorne o resultado.

Exemplo:

Entrada: `A=[[1,2],[3,4]], B=[[2,0],[1,2]]`

Saída: `[[4,4],[10,8]]`

14. Triângulo de Pascal

https://pt.wikipedia.org/wiki/Tri%C3%A2ngulo_de_Pascal

Crie uma função `triangulo_pascal(n)` que retorne uma lista de listas representando as primeiras `n` linhas do triângulo de Pascal.

Exemplo:

Entrada: 5

Saída: `[[1],[1,1],[1,2,1],[1,3,3,1],[1,4,6,4,1]]`

15. Sudoku – Linhas Únicas

Implemente `linhas_validas(sudoku)` que receba uma matriz 9x9 e verifique se todas as linhas contêm apenas números de 1 a 9 sem repetição.

16. Espiral de Números

Escreva uma função `matriz_espiral(n)` que construa uma matriz $n \times n$ preenchida com números de 1 a n^2 , em ordem espiral.

Exemplo:

Entrada: 3

Saída:

[[1,2,3],[8,9,4],[7,6,5]]

17. Rotação de Lista

Implemente `rotacionar(lista, k)` que rotacione os elementos de uma lista k posições à direita usando apenas laços.

Exemplo:

Entrada: ([1,2,3,4,5], 2)

Saída: [4,5,1,2,3]

18. Combinações de Pares

Crie uma função `pares(lista)` que receba uma lista de números e retorne uma lista com todos os pares possíveis (sem repetição).

Exemplo:

Entrada: [1,2,3]

Saída: [(1,2),(1,3),(2,3)]

19. Flatten de Lista Aninhada

Escreva `flatten(lista)` que receba uma lista com sublists aninhadas de profundidade arbitrária e retorne uma lista "achatada".

Exemplo:

Entrada: [1, [2, [3, 4]], 5]

Saída: [1,2,3,4,5]

20. Produto Cartesiano

Implemente `produto_cartesiano(lista1, lista2)` que retorne todas as combinações possíveis entre elementos de duas listas.

Exemplo:

Entrada: ([1,2], ['a','b'])

Saída: [(1,'a'),(1,'b'),(2,'a'),(2,'b')]

20. Sequencia de collatz

A conjectura de Collatz é uma conjectura matemática que recebeu este nome em referência ao matemático alemão Lothar Collatz. Esta conjectura aplica-se a qualquer número natural inteiro, e diz-nos para, se este número for par, o dividir por 2 e se for ímpar, para multiplicar por 3 e adicionar 1. Embora não haja uma prova, acredita-se que sempre o valor vai chegar a 1.

Crie a função `collatz(num)` que retorna uma lista com todos os números da sequência até chegar a 1.

Exemplo:

```
collatz(127)
```

O maior número é 4372 e tem 47 itens.

```
[127, 382, 191, 574, 287, 862, 431, 1294, 647, 1942, 971, 2914, 1457, 4372, 2186, 1093, 3280, 1640, 820, 410, 205, 616, 308, 154, 77, 232, 116, 58, 29, 88, 44, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1]
```

21. Nosso Split

O `split` é um método interno do Python que visa separar um string em uma lista de partes, segundo um substring como separador. Veja um exemplo de código:

```
1 frase = "Exemplo de frase para usar com split."
2 minha_lista = frase.split()
3 print(minha_lista)
4 # resultado: ['Exemplo', 'de', 'frase', 'para', 'usar', 'com', 'split.']
5
6 # exemplo 2
7 # split com separador especificado
8 frase = "O evento ocorreu no dia 01/01/2024"
9 minha_lista = frase.split("/")
10 print(minha_lista)
11 # resultado: ['O evento ocorreu no dia 01', '01', '2024']
```

Crie a função `NossoSplit(string,separador)`, que funciona como o `split` nativo do Python. Você deve receber como parâmetro o string e o separador e retornar a lista adequada. Observe que na implementação desta função você não pode utilizar `split` nem quaisquer funções especiais de string do Python.

22. Nosso Replace

O método `replace()` em Python substitui todas as ocorrências de uma substring por outra em uma string. A sintaxe é `string.replace(antiga, nova, [contagem])`, onde `antiga` é o texto a ser substituído, `nova` é o texto de substituição e `contagem` (opcional) limita o número de substituições. `ser substituído`, `nova` é o texto de substituição e `contagem` (opcional) limita o número de substituições. Veja os exemplos:

Python

```
# Exemplo básico: Substituir todas as ocorrências de uma palavra
frase = "Eu gosto de comer pizza no almoço."
nova_frase = frase.replace("pizza", "hambúrguer")
print(nova_frase)
# Saída: Eu gosto de comer hambúrguer no almoço.
```

Python

```
# Exemplo com a contagem de ocorrências:
# Substituir apenas as duas primeiras ocorrências de "a" por "o"
s = "banana"
s_modificada = s.replace("a", "o", 2)
print(s_modificada)
# Saída: bonona
```

Crie a função `NossoReplace(String,original,mudar,qtde)` e deve retornar a string atualizada. Na implementação desta função, você só pode utilizar as funções básicas do Python.